CPT204 Mock Exam - Question 13
OnlineCourse

Given the following interface:

```
interface Payable {
    double getPaymentAmount();
}
```

Complete the class OnlineCourse that implements Payable.

An online course has:

- courseName
- baseFee
- level
- hasCertificate

The course level surcharge rules are:

- BEGINNER: +0
- INTERMEDIATE: +100
- ADVANCED: +200

If hasCertificate is true, add an extra 50 after adding the level surcharge.

If baseFee is negative, getPaymentAmount() should return 0.

Answer:

```
interface Payable {
    double getPaymentAmount();
}

class OnlineCourse implements Payable {
    private String courseName;
    private double baseFee;
    private Level level;
    private boolean hasCertificate;

    public enum Level {
        BEGINNER, INTERMEDIATE, ADVANCED
    }

    public OnlineCourse(String courseName, double baseFee,
                        Level level, boolean hasCertificate) {
        // TODO: Please complete this method.
    }

    @Override
    public double getPaymentAmount() {
        // TODO: Please complete this method.
    }
}
```

CPT204 Mock Exam - Question 14
KeywordCounter

Given the following interface:

```
interface Countable {
    int getKeywordCount();
}
```

Complete the class KeywordCounter that implements Countable.

A KeywordCounter has:

```
private String text;
private HashSet<String> keywords;
private HashMap<String, Integer> wordCounts;
```

The class counts how many times selected keywords appear in a piece of text.

The input text will contain lowercase words separated by single spaces only. There will be no punctuation.

Rules:

- The constructor should initialise all properties.
- Store all given keywords in the HashSet.
- Count all words in the text using the HashMap.
- getKeywordCount() should return the total number of times any keyword appears in the text.
- If text is null or empty, return 0.

Answer:

```
import java.util.*;

interface Countable {
    int getKeywordCount();
}

class KeywordCounter implements Countable {
    private String text;
    private HashSet<String> keywords;
    private HashMap<String, Integer> wordCounts;

    public KeywordCounter(String text, Collection<String> keywords) {
        // TODO
    }

    private void countWords() {
        // TODO
    }

    @Override
    public int getKeywordCount() {
        // TODO
    }
}
```

CPT204 Mock Exam Questions 13-14 - Completed Suggested Answers

Question 13 - OnlineCourse

```
interface Payable {
    double getPaymentAmount();
}

class OnlineCourse implements Payable {
    private String courseName;
    private double baseFee;
    private Level level;
    private boolean hasCertificate;

    public enum Level {
        BEGINNER, INTERMEDIATE, ADVANCED
    }

    public OnlineCourse(String courseName, double baseFee,
                        Level level, boolean hasCertificate) {
        this.courseName = courseName;
        this.baseFee = baseFee;
        this.level = level;
        this.hasCertificate = hasCertificate;
    }

    @Override
    public double getPaymentAmount() {
        if (baseFee < 0) {
            return 0;
        }

        double amount = baseFee;
        if (level == Level.INTERMEDIATE) {
            amount += 100;
        } else if (level == Level.ADVANCED) {
            amount += 200;
        }

        if (hasCertificate) {
            amount += 50;
        }
        return amount;
    }
}
```

Question 14 - KeywordCounter

```
import java.util.*;

interface Countable {
    int getKeywordCount();
}

class KeywordCounter implements Countable {
    private String text;
    private HashSet<String> keywords;
    private HashMap<String, Integer> wordCounts;

    public KeywordCounter(String text, Collection<String> keywords) {
        this.text = text;
        this.keywords = new HashSet<>(keywords);
```

```
        this.wordCounts = new HashMap<>();
        countWords();
    }

    private void countWords() {
        if (text == null || text.isEmpty()) {
            return;
        }

        String[] words = text.split(" ");
        for (String word : words) {
            if (wordCounts.containsKey(word)) {
                wordCounts.put(word, wordCounts.get(word) + 1);
            } else {
                wordCounts.put(word, 1);
            }
        }
    }

    @Override
    public int getKeywordCount() {
        if (text == null || text.isEmpty()) {
            return 0;
        }

        int total = 0;
        for (String keyword : keywords) {
            if (wordCounts.containsKey(keyword)) {
                total += wordCounts.get(keyword);
            }
        }
        return total;
    }
}
```